Dialback

BUILD PLAN · NOV 9, 2026 · FOR THE NOV 19 LIVE FINAL

Building the mapped screens on today's backend

The wireframes map 12 product screens. Nine are on main as a restyle (spec 0041, PR #95). This plan covers what's left: three pages that map and present the product, the seven-station rail, and the places where screens need data the API doesn't send yet. That data is already in Postgres, except per-turn call answers and lab results.

16
wireframe boards on 5 canvas pages

9 of 12
product screens already on main

3
new pages, web only, no contract change

8 of 10
data gaps readable from data we already store

1. Where the mapped screens are

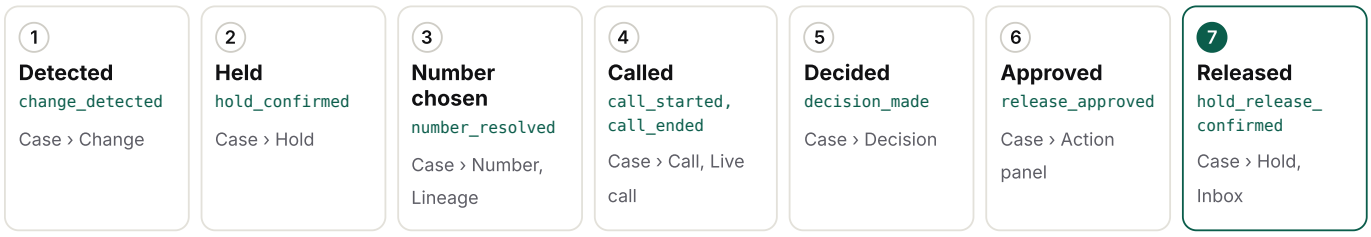
All the screens exist in the **Dialback wireframes** canvas. It opens on its first page, *Signature directions*, which only holds the four brand explorations. Switch pages at the top of the canvas to see the rest.

Canvas page	Boards	What it is for
Signature directions	A Callback rings, B The rail, C The proof, D Sound waves	Visual language. A shipped in spec 0041.
System map	System map	The product on one page: 7 stations, data source, screen. Not built yet.
AP app	Inbox, Case detail (Acme, as Taylor), Action panel states, Number lineage, Mobile	Clerks and approvers move each change to release.
Admin console	Overview, Trust Ledger, Evidence (tampered), Assurance	Controllers and auditors see posture, integrity and testing.
Demo tools	Live call split view, Flow mode	Projector views for the live demo. Not built yet.

Each board carries purple sticky notes with its data source and demo timing. They agree with this plan.

2. The product map: seven stations, one screen each

Every bank change moves through the same seven stations. Each station is already an event in the evidence chain. So the rail, the system map, the live call view and flow mode can all be drawn from data the API sends today.



Around the rail: Posture (Trust Ledger) before any change; Record (Evidence) and Test (Assurance) across every case.

How each station reads, from the timeline of the case's current version

Station	Done when this version has	Current while the state is	Branch marker (amber or red), with the label
1 Detected	`change_detected` (always, once a case exists)	`open`	none
2 Held	`hold_confirmed`	`held`, before a number	`hold_failed`: "Hold not confirmed"
3 Number chosen	`number_resolved`	`held`, after the hold	`no_provable_number`: "No provable number"
4 Called	`call_ended` that moved to `deciding`	`calling`, `retry_wait`	`call_timeout` or tries used up: "No person reached"
5 Decided	`decision_made`	`deciding`	outcome `not_verified`: "Not verified" (red); `needs_review`: "Needs review"
6 Approved	`release_approved`	`awaiting_release`, `needs_review` after a decision	`release_co_approved`: "1 of 2"; `release_rejected` or `review_blocked`: "Rejected" (red)
7 Released	`hold_release_confirmed`	`releasing`	none

The current version's events are the ones after the case's latest `change_detected`. A `deadline_reached` or `command_failed` puts the amber marker on whichever station is current. The case page reads `timeline[]`. The inbox has no timeline, so it reads `GET /api/datasets/{id}/evidence` (each row has `case_id` and `type`) and groups the rows by case. No new data is needed for either.

3. Status of every screen against main

"On main" means after PR #95 (spec 0041). Phases: **1** is web only with no contract change; **2** needs a small additive read in the API; **3** needs data that doesn't exist yet.

Screen	Route	On main	What is still missing	Phase
System map	`/d/[id]/map` (new)	Not built	The whole page: the live rail with a count at each station, the station table with links into the demo cases, exception paths with live examples, surfaces.	1
Inbox	`/d/[id]/cases`	Built	7-dot rail (5 today). Dollar amounts (bill counts stand in). Sentences in the activity feed.	1, 2
Case detail	`/d/[id]/cases/[caseId]`	Built	7-station stepper. Refresh while the case is moving. Money and invoice numbers. The change-request email.	1, 2
Action panel states	on the case page	Built	Nothing; states A to G come from `permissions` and the core's refusals.	done
Number lineage	`…/[caseId]/lineage`	Approximate	The chart draws one dot per candidate per source type at its first-seen date. The board shows every sighting.	2
Mobile	responsive	Built	Nothing.	done
Overview	`/d/[id]/overview`	Partial	The health row shows the chain, the last check and alerts. Missing: poller, nightly sweep, reconciliation, webhooks and bad signatures, Ramp mode, voice service.	2
Trust Ledger	`/d/[id]/risk`	Built	The vendor drill-in: documents with custody, hidden-text flags and the numbers found in each.	2
Evidence	`/d/[id]/evidence`	Built	A one-line summary per row; filters by type and vendor.	2
Assurance	`/corpus`, `/scorecard`	Built	The corpus is real. The scorecard stays an empty state until the attack lab is on main (spec 0048).	3
Live call	`…/[caseId]/call` (new)	Not built	The whole page. Answers arrive together at the end of the call today; true turn by turn is phase 3.	1, 3
Flow mode	`…/[caseId]/flow` (new)	Not built	The whole page.	1
Rules & policy	`/d/[id]/rules` (new)	Not boarded	In the design prompt, not on the canvas: R1 to R9, the rule table, selection settings, call policy.	2
People, Integrations	none	Not boarded	Fold Integrations into the Overview health row. People is optional.	3

4. The data each gap needs, and where it already lives

The design asked for things the API doesn't send. For eight of ten, the backend already stores the data and only a read is missing. Every change below is **additive and optional** in the contract, so older cores and pages keep working.

Gap	Where the data is today	Expose as	Owner
G1 Money and invoice numbers	In Ramp: `mock_ramp.bills` has `amount_cents`, `currency_code`, `invoice_number`. The core's `RampBill` drops them and the hold saves only `bill_id` and `payment_date`.	Add the three fields to `RampBill` (confirm Ramp's Bill API names). Save them with the schedule at hold time in a new nullable column on `app.holds`, outside the hashed evidence body. API: optional `HoldInfo.bills[]` and `CaseSummary.held_cents`.	Abhi

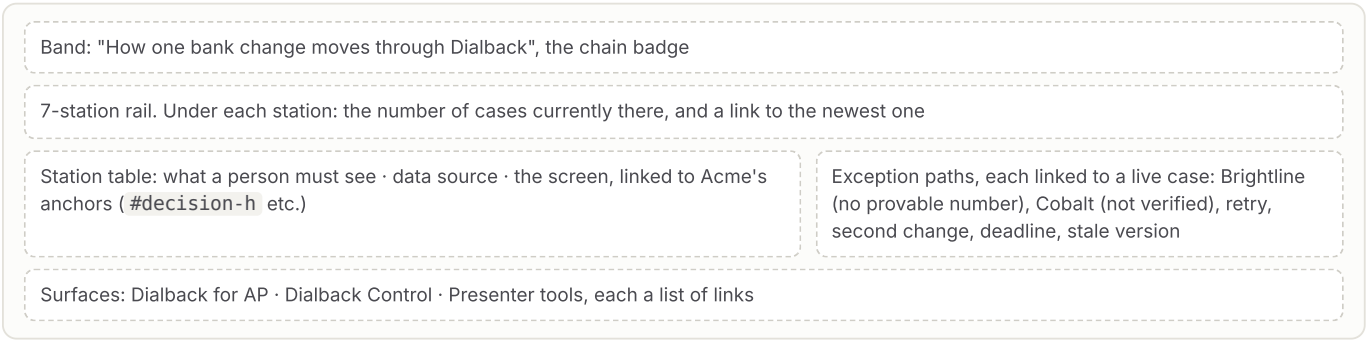
Gap	Where the data is today	Expose as	Owner
G2 The change-request email	`ledger.change_requests` → `ledger.documents` (sender, subject, thread, received_at, channel, sha256) and `ledger.observations` (phone numbers with their quote).	Optional `CaseDetail.request`: sender, subject, thread, received time, channel, sha256, and the numbers found in it, each marked with whether the selection refused it. No body text.	Inesh, Abhi
G3 Every sighting for the lineage chart	`ledger.observations`: value, source_type, seen_at, role, visibility, document. `vendor_rows()` already reads them as of a time.	Optional `SelectionView.sightings[]`: e164, source type, seen_at, role, visibility, in change context. Only rows recorded before the case opened, as the selection rule reads them.	Inesh
G4 Vendor documents (Trust Ledger drill-in)	`ledger.documents` and `ledger.observations`.	New `GET /api/datasets/{id}/vendors/{vendor_id}/documents`: kind, channel, received_at, claimed_date, sender, subject, sha256, hidden-text flag, numbers with role and quote.	Inesh
G5 Control health	`app.job_runs` (poll, full_sweep, reconcile: last run, ok, error), `app.inbound_webhooks` (count, `signature_valid`), the verify job's last check (sent today), the `DIALBACK_*` flags, `VOICE_URL`.	New `GET /api/datasets/{id}/health`. The voice row can be "configured" or a server-side `/healthz` ping with a short timeout.	Abhi
G6 A sentence per evidence row	The core's timeline templates already write `TimelineEntry.summary`.	Optional `EvidenceRow.summary`, from the same templates.	Abhi
G7 Rules & policy	Code: the rule table in `packages/decision` (id, block, sentence, engine version), the ledger's rule settings, the call policy's window and caps.	A generated `apps/web/src/data/rules.json` from a script with a CI freshness check, as `corpus-results.json` is made today. No endpoint needed.	Abhi, Sami
G8 People and roles	`app.user_profiles`, `app.user_roles`.	Optional `GET /api/people` for approvers and auditors. Low priority.	Abhi
G9 Live answers, turn by turn	Not stored `voice.call_answers` exists, but nothing writes to it, and `status` never reaches `in_progress`. Answers travel only inside `call_ended`.	The voice service writes each recorded answer as it lands (count only for the last four) and sets `in_progress` on connect. Optional `CallAttempt.live`. The decision still reads only `call_ended`.	Sami
G10 The attack-lab scorecard	Not on main On `sami/attack-lab-*`; spec 0048 lands it.	A script copies the lab's `scorecard.json` into `apps/web/src/data/`; the page renders gates and rates. Never sample numbers.	Sami

Fix the seed with G2. The Ramp mock's Acme change ends in 0001 (`scripts/demo/seed.json`), but the ledger's request email for all three vendors says "account ending 4821" (`packages/ledger/…/demo_seed.json`). G2 shows no body text, so the screen won't contradict itself. Fix the generator anyway (`corpus/generator/demo.py`, then `build.py`) so the email matches each vendor's change.

5. The three new pages (phase 1, no new data)

5.1 System map: `/d/[datasetId]/map`

This is the page that maps the product. It is the board *System map* made live, the first item under Control in the sidebar, and the first step of the guided demo.



Data: `core.allCases(datasetId)` (released cases included) and `core.evidence(datasetId)`, through the station function from section 2. The station table and the surface descriptions are fixed product copy, like the landing page. Nothing on the page explains a decision.

Empty state: with no cases, the rail shows zeros and the page says the team starts a change with `dialback-core demo change`.

5.2 Live call: `/d/[datasetId]/cases/[caseId]/call`

This is the centerpiece of demo beat 4, and the board *Live call split view*. On the left, a phone frame: the number dialed (`selection.e164`), the customer's name, a timer from the current call's `started_at`, and the question being asked. On the right: the disclosure (`call_script.disclosure`), then the seven slots with their question wording (`call_script.questions`) and the recorded answers. Along the top: the vendor, the state, "attempt n of 3", and the 7-station rail.

What the API shows	What the page shows
State `held`, no call yet	"Choosing a number" or "Dialing (206) 555-0142"; slots empty.
A call with `started_at` and no `result`	"On the call · 01:42"; the disclosure shown as played; slots say "Answers arrive when the call ends." Honest about phase 1.
The call ended with `answers_recorded`	Answers fill in turn order with a short fade (none under reduced motion). Last four: "4 digits given · match". Script-check chips appear.
`no_answer`, `voicemail`, `callback_requested`, `declined`	The core's `call.reason`, and "Retry scheduled" with the attempt.
`decision_made`	The outcome badge, the core's `effect`, and a link to the case's decision.

Refresh: a small client component asks a route handler in the same folder every 2 seconds. The handler is server-side and calls `core.case(caseId)` with the person's own token. The browser still never calls the core. Polling stops once the case leaves `calling`, `retry_wait` and `deciding`. Phase 3 (G9) swaps the "Answers arrive when the call ends" state for real per-turn answers, with no layout change.

5.3 Flow mode: `/d/[datasetId]/cases/[caseId]/flow`

The board *Flow mode, projector*: a 16:9 dark view. Headline "AI collects. Code decides. A person releases." Seven columns, one per station. Each lights up when its event lands, showing its time, the core's `TimelineEntry.summary` and the short hash. The chain badge sits in the corner. It uses the same refresh as the live call. Use it to close the demo, or keep it on a second screen during the live call.

5.4 Small changes that come with phase 1

- **Stepper and mini rail:** the case stepper and the inbox mini rail switch to seven stations. Keep `progress.ts` and its tests for anything that still uses five stages, until nothing does.
- **Case page refresh:** the case page uses the same 2-second refresh while the state is `open`, `held`, `calling`, `retry_wait`, `deciding` or `releasing`, so the stepper moves on its own during the demo.
- **Guided demo:** add System map as the first step. Point "The call" at the live call page. Keep `?tour=N` and the 8-step tests passing, or update them in the same PR.
- **Links:** the case's Call card links to the live call page, and its header links to flow mode.

6. Order of work

Each row is one spec in `specs/` and one or more small PRs. Specs 0042 to 0048 are taken on Sami's branches, so new work starts at **0049**. Check `git branch -r` before claiming a number. A contract change goes in its own PR, approved by both other teammates.

Spec	Work	Touches	Owner	When
Phase 1: web only, no contract change. Can land before the Nov 10 submission if the team wants; otherwise first thing Nov 11.				
0049	Seven stations: `lib/stations.ts` with tests, the case stepper, the inbox mini rail, the case page refresh	`apps/web`	Abhi	Nov 10–11
0050	System map page, the nav item, tour step 1	`apps/web`	Abhi	Nov 11
0051	Presenter views: live call and flow mode, the polling route handler, tour retarget	`apps/web`	Abhi, Sami	Nov 11–12
Phase 2: additive reads. Each item is a contracts PR, then a core or ledger PR, then a web PR.				
0052	G1 Money and invoice numbers on holds and the queue	contracts, core, mock, web	Abhi	Nov 12–13
0053	G2 The change request on the case page, and the seed fix	contracts, ledger, core, corpus, web	Inesh, Abhi	Nov 12–13
0054	G3 Lineage sightings	contracts, ledger, web	Inesh	Nov 13–14
0055	G5 Control health	contracts, core, web	Abhi	Nov 13–14
0056	G6 Evidence row summaries and client-side filters	contracts, core, web	Abhi	Nov 14
0057	G7 Rules & policy page from generated `rules.json`	scripts, web, CI	Abhi, Sami	Nov 14–15
0058	G4 Vendor documents drill-in on the Trust Ledger	contracts, ledger, web	Inesh	Nov 15
Phase 3: new data. Only after the team says yes.				
0059	G9 Live answers turn by turn	voice, contracts, core, web	Sami	Nov 15–17
after 0048	G10 Scorecard on Assurance	evals, scripts, web	Sami	when the lab merges
optional	G8 People page	contracts, core, web	Abhi	if time
Nov 18: full rehearsal on the deployed stack, then record backup clips of every beat. **Nov 19:** live final.				

7. The demo, beat by beat, on these routes

Time	Beat	Route	What makes it land	Needs
0:00–0:20	The problem	`/d/{id}/map`	The whole flow on one screen; Brightline and Cobalt already counted at their stations	0050
0:20–0:45	Before any change	`/d/{id}/risk`	Cobalt's near miss; Brightline can't be proven	main
0:45–1:10	A change, held	`/d/{id}/cases` → Acme	`demo change --vendor acme`; the row appears and the stepper moves on its own	0049
1:10–1:35	Why this number	`…/acme/lineage`	Three years of sightings; Cobalt's 0143 refused	main (0054 better)
1:35–2:35	The live call	`…/acme/call`	Phone frame, disclosure first, answers land, "4 digits given"	0051
2:35–2:55	Code decides	`…/acme#decision-h`	12 of 12 conditions, in the rule table's words	main
2:55–3:25	Four eyes	case action panel, "Viewing as"	Pat is refused by Postgres; Taylor releases; dates come back	main
3:25–3:50	Tamper	`/d/{id}/evidence`	`demo tamper`; the chain names the row	main
3:50–4:30	Proof it holds	Brightline, Cobalt, `/corpus`	No number means no call; a denial means blocked; 20 of 20 frozen cases	main
4:30–5:00	Close	`…/acme/flow`	Seven stations lit, each with its hash	0051

Stage Brightline and Cobalt with `demo change` before going on stage; run Acme live.

8. Rules for whoever builds this

These come from `AGENTS.md`, `apps/web/AGENTS.md` and spec 0041. They outrank this plan.

- **The browser never calls the core.** Pages and route handlers fetch server-side through `src/lib/core.ts` with the signed-in person's token. Polling goes through a route handler, never straight to the core or to Supabase.
- **Every sentence that explains a decision, refusal, reason, status or effect is the core's text** (`effect`, `explanation`, checklist `sentence`, `permissions.reasons`, `summary`, `status_text`, `reason_text`, `notes`, `Alert.message`, `call.reason`, `call_script.*`). Say "provable" or "not provable", never "safe".
- **No data the API didn't send.** No invented amounts, invoice numbers, health or scores. A screen waiting on a phase 2 field shows what exists today (bill counts, for example) until the field lands.
- **ADR-8 on screen:** routing number and last four only, formatted `026009593 ····0001`; spoken digits only as "4 digits given" plus the result.
- **Permissions only hide buttons.** The database refuses the changer, the reviewer and a stale version. "Viewing as" signs in as the demo accounts (spec 0034); it never fakes a role in the browser.
- **Contracts:** never edit `packages/contracts/generated/**`; change the schema, then regenerate. Contracts only grow (new optional fields and endpoints), each change in its own PR.
- **Money-path SQL, the selection rule and the decision engine** are planned in plan mode first. A new read never writes to `app.cases`, `app.decisions` or `app.releases`.
- **Keep** every anchor (`#decision-h`, `#row-{seq}`, `#rule-{id}`), `?tour=N`, `aria-*` and focus rings. Nothing moves under `prefers-reduced-motion`.
- **Process:** spec from `specs/TEMPLATE.md` first, failing tests next, PRs under about 400 changed lines, the reviewer agent on every diff, the commands' output pasted into the PR. Agents open PRs and never merge.

9. Done means

- Every board on the AP app, Admin console, Demo tools and System map pages has a route, and each matches its board at 1440 and 390 px (flow mode at 16:9).
- The demo runs start to finish on the routes in section 7, on a fresh `demo reset`, without touching the address bar except to start.
- Each phase 2 field is optional in the contract, and each page still renders when the field is missing (an older core).
- These pass on every PR: `pnpm check:api && pnpm typecheck && pnpm test && pnpm build`, `python3 scripts/ci/boundaries.py`, plus `scripts/db/test.sh` and the owner's pytest when the core, ledger or voice changed.
- No screen shows a full account number, spoken digits, a sentence the dashboard composed to explain a decision, or a number the system didn't compute.

The companion prompt, `dialback-ui-build-prompt.md`, hands this plan to a coding agent one phase or item at a time.
Sources in the repo: `specs/0041-ui-refresh.md`, `docs/ui-implementation-prompt.md`, `packages/contracts/openapi/dialback.yaml`, `supabase/migrations/{app,ledger,voice,mock_ramp}`, `apps/core/src/dialback_core/ramp/port.py`, `docs/state-machine.md`, the wireframes canvas notes. Branch facts as of Nov 9, 2026, 11 PM PT.